Team26_DESIGN_DOC.md

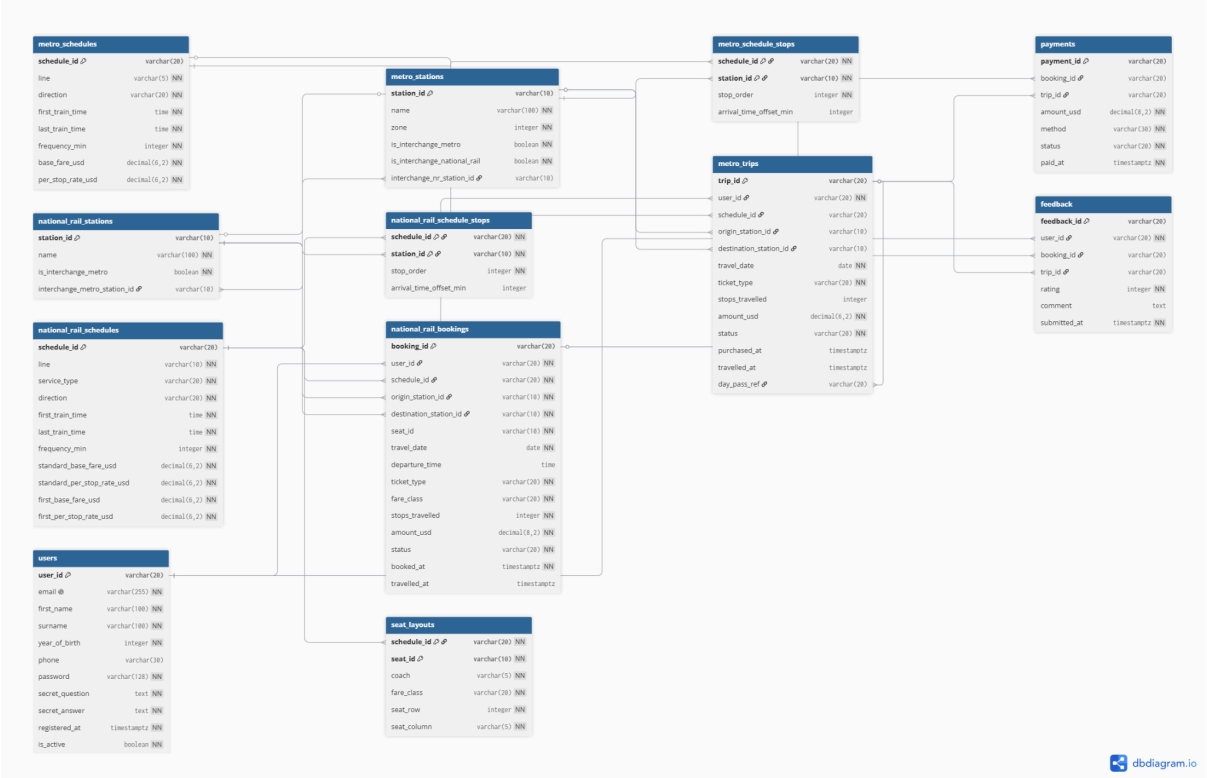
TransitFlow — Database Design Document

Team 26

Name	Student ID	Role
廖可筠	113403014	Relational Database Architect (PostgreSQL)
羅乃慈	113403533	Graph Database Engineer (Neo4j)
沈昕慧	113403010	AI Integration & Vector Database (pgvector)

Section 1 — Entity-Relationship Diagram

1.1 ER Diagram



1.2 Cardinality Summary

Relationship	Cardinality	Notes
metro_schedules → metro_schedule_stops	1:N	One schedule has many stops
metro_stations → metro_schedule_stops	1:N	One station appears in many schedules
national_rail_schedules → national_rail_schedule_stops	1:N	Same pattern as metro
national_rail_schedules → seat_layouts	1:N	One schedule has many seats
users → national_rail_bookings	1:N	One user has many bookings
users → metro_trips	1:N	One user has many trips
national_rail_bookings → payments	1:1	One booking has one payment
metro_trips → payments	1:1	One trip has one payment
metro_trips → metro_trips (day_pass_ref)	1:N	Self-referencing for day pass
metro_stations ↔ national_rail_stations	1:1	Interchange stations

---

Section 2 — Normalisation Justification

2.1 Third Normal Form (3NF) Design Decision

The most important normalisation decision in this schema is placing **schedule stops in a separate junction table** rather than storing them as an array column.

Before normalisation (violates 1NF and 3NF):

```
-- WRONG: stops stored as an array in the schedule row
CREATE TABLE metro_schedules (
    schedule_id  VARCHAR(20) PRIMARY KEY,
    line         VARCHAR(5),
    stops        JSONB  -- ["MS01", "MS02", "MS09"] -- violates 1NF
);
```

This design violates **First Normal Form (1NF)** because a single column (`stops`) holds multiple values. It also creates a **transitive dependency**: the stop order depends on the station ID, not directly on the schedule ID, violating **3NF**.

After normalisation (3NF compliant):

```
CREATE TABLE metro_schedule_stops (
    schedule_id  VARCHAR(20) REFERENCES metro_schedules(schedule_id),
    station_id   VARCHAR(10) REFERENCES metro_stations(station_id),
    stop_order   INTEGER NOT NULL,
    PRIMARY KEY (schedule_id, station_id)
);
```

Now each fact lives in exactly one place:

- `metro_schedules` stores schedule-level data (line, fares, frequency)
- `metro_schedule_stops` stores the ordered stop sequence
- The functional dependency is `(schedule_id, station_id) → stop_order`, which is a full dependency on the composite key — satisfying **2NF**
- There are no transitive dependencies — satisfying **3NF**

This design enables efficient queries like "find all schedules where MS01 comes before MS09" using a simple JOIN with a `WHERE stop_order < dest_stop_order` condition.

2.2 Deliberate De-normalisation Trade-off

One deliberate de-normalisation exists in `national_rail_bookings`: we store `amount_usd` directly in the booking row rather than computing it from the schedule's fare rates at query time.

Reason: Fare rates may change after a booking is made. Storing the amount at booking time creates a **snapshot** of the fare the passenger actually paid. Computing it dynamically would produce incorrect results if rates were ever updated.

This violates strict 3NF (amount depends on fare class and stops, which are also stored in the row), but the trade-off is intentional and justified by business requirements.

2.3 Password Hashing

We use **bcrypt** for password storage, implemented via the `bcrypt` Python library.

Algorithm choice: bcrypt was chosen over MD5 or SHA-1 because:

- MD5 and SHA-1 are designed to be *fast*, making them vulnerable to brute-force attacks. A modern GPU can compute billions of SHA-1 hashes per second.
- bcrypt is deliberately *slow* and has an adjustable cost factor (`rounds=12` in our implementation), meaning the computation time increases as hardware improves.

- bcrypt uses an adaptive hashing approach — each hash takes approximately 250ms to compute, making brute-force attacks computationally infeasible.

Salt management: bcrypt automatically generates a unique random salt for each password using `bcrypt.gensalt()`. The salt is embedded in the stored hash string (e.g., `$2b$12$X9v...`). This means:

- Two users with the same password will have completely different hash values
- Rainbow table attacks are defeated because pre-computed hash tables cannot account for the unique salt
- No separate salt column is needed — the salt is part of the hash string itself

```
# Registration: hash with auto-generated salt
hashed = bcrypt.hashpw(password.encode("utf-8"), bcrypt.gensalt()).decode("utf-8")

# Login: verify by re-hashing with the embedded salt
is_valid = bcrypt.checkpw(password.encode("utf-8"), stored_hash.encode("utf-8"))
```

2.4 Database Terminology Used

- **Functional dependency**: `schedule_id → line, direction, base_fare_usd` (schedule attributes depend on the schedule ID)
- **Candidate key**: `email` in `users` is a candidate key (unique, not null) alongside the primary key `user_id`
- **Transitive dependency**: storing `station_name` in `bookings` would create a transitive dependency (`booking_id → station_id → station_name`), which we avoid by using a FK join instead
- **Composite key**: `(schedule_id, station_id)` in junction tables, where neither column alone is unique

---

Section 3 — Graph Database Design Rationale

3.1 What is Stored as Nodes, Relationships, and Properties

Nodes represent entities with independent identity — things that exist on their own, regardless of their connections:

- `MetroStation {station_id, name, lines}` — each station is a distinct entity that persists even if no trains currently run through it. Its identity comes from `station_id`, not from which routes it serves.
- `NationalRailStation {station_id, name, lines}` — same reasoning applies. A rail station is its own entity in the physical world.

Relationships represent connections that only exist *between* entities — they have no independent existence:

- `METRO_LINK` — a physical track segment between two adjacent metro stations. This connection cannot exist without both endpoints; modelling it as a node would be wrong (there is no "Segment entity").
- `RAIL_LINK` — same reasoning for national rail segments.
- `INTERCHANGE_TO` — a walking connection between a metro station and a rail station at the same physical location. It is a fact about their relationship, not a thing in itself.

Properties store attributes that belong intrinsically to the node or relationship:

- Node properties: `station_id`, `name`, `lines` — these describe the station itself.
- Relationship properties: `line`, `travel_time_min` — these describe the specific connection (which service, how long it takes). Crucially, `travel_time_min` on each relationship serves as the **edge weight** for Dijkstra's algorithm, enabling weighted shortest-path queries without joining any external table.

3.2 Why a Graph Database is Better than Relational for Routing

In a relational database, finding the shortest path requires a recursive CTE:

sql

```
WITH RECURSIVE path AS (
    SELECT station_id, ARRAY[station_id] AS visited, 0 AS total_time
    FROM metro_stations WHERE station_id = 'MS01'
    UNION ALL
    SELECT l.to_station_id, p.visited || l.to_station_id, p.total_time + l.travel_time_min
    FROM path p
    JOIN metro_links l ON l.from_station_id = p.current
    WHERE l.to_station_id <> ALL(p.visited)
)
SELECT * FROM path WHERE station_id = 'MS14' ORDER BY total_time LIMIT 1;
```

This query is complex, hard to maintain, and does not scale. PostgreSQL must materialise the entire recursive result set and cannot run Dijkstra's algorithm natively — it expands all possible paths breadth-first, which grows exponentially.

In Neo4j with APOC, the same query is:

cypher

```
MATCH (start:MetroStation {station_id: "MS01"}),
    (end:MetroStation {station_id: "MS14"})
CALL apoc.algo.dijkstra(start, end, "METRO_LINK", "travel_time_min")
YIELD path, weight
RETURN path, weight
```

Neo4j stores data as a **native graph** — each node holds direct pointers to its adjacent nodes. Traversal is O(log n) with graph indices because the engine follows physical pointers rather than scanning rows. For our delay ripple analysis (finding all stations within N hops), the graph traversal is a single Cypher statement, whereas SQL requires N self-joins or a recursive CTE that grows exponentially in complexity.

3.3 Query Types Enabled by the Graph Model

Query 1: Shortest path (`query_shortest_route`) — The `METRO_LINK` and `RAIL_LINK` relationships with `travel_time_min` weights enable Dijkstra's algorithm as a native operation via APOC. No joins, no recursion.

Query 2: Cross-network interchange path (`query_interchange_path`) — The `INTERCHANGE_TO` relationship connects `MetroStation` nodes directly to `NationalRailStation` nodes. A single Cypher `shortestPath` call can traverse `METRO_LINK → INTERCHANGE_TO → RAIL_LINK` in one statement. In SQL this would require a three-way JOIN across two separate station tables plus a union — and it still could not express the path-finding logic cleanly.

Query 3: Alternative routes avoiding a station (`query_alternative_routes`) — The `NONE(n IN nodes(path) WHERE n.station_id = $avoid)` Cypher clause filters out paths containing a specific node during traversal. There is no SQL equivalent without complex correlated subqueries.

Query 4: Delay ripple (`query_delay_ripple`) — `MATCH (s)-[:METRO_LINK|RAIL_LINK*1..N]-(affected)` finds all stations within N hops in one statement, regardless of network. SQL would require N self-joins, with a new join added every time you want to extend the ripple by one more hop.

3.4 Node Identity

Each node is uniquely identified by `station_id` (e.g., `MS01`, `NR05`). We chose `station_id` as the identity property because:

- It is already the natural primary key in the PostgreSQL schema, ensuring **consistency across both databases** — a station referred to as `NR01` in SQL is the

same node as `{station_id: "NR01"}` in Neo4j, with no ID mapping layer needed.
- It is **human-readable**, making debugging and Cypher query writing easier.
- It is **stable** — station IDs do not change even if the station's name is updated (e.g., a renamed station keeps `MS07` even if its display name changes).

Uniqueness is enforced by constraints in `seed.cypher`:

cypher

```
CREATE CONSTRAINT metro_station_id_unique
FOR (s:MetroStation) REQUIRE s.station_id IS UNIQUE;
```

Section 4 — Vector / RAG Design

4.1 What is Embedded and Why Cosine Similarity

We embed **policy documents** — refund policies, booking rules, ticket types, and travel conduct policies — stored in the `train-mock-data/` JSON files. Each document is converted to a 768-dimensional vector and stored in the `policy_documents` table alongside the original text.

Cosine similarity is appropriate for semantic search because it is **magnitude-independent**: it measures the angle between two vectors in high-dimensional space, not their absolute length. This means a short query like "refund delay" and a long policy document about delay compensation will still match closely if they discuss the same concept, even though the document vector has larger magnitude due to more words.

In contrast, **Euclidean distance (L2)** would penalise documents simply for being longer, regardless of semantic content. A 500-word policy about refunds would appear "farther" from a short query than a 10-word sentence on the same topic, which is not the behaviour we want.

The database stores vectors as `vector(768)` and uses the `<=>` cosine distance operator provided by pgvector.

4.2 The Full RAG Pipeline

Stage 1 — Query embedding: The user's question (e.g., "Can I get compensation for a 45-minute delay?") is converted to a 768-dimensional vector using the `nomic-embed-text` Ollama model via `llm.embed(user_question)`. The same model used to embed the documents at seed time must be used here — mixing models produces meaningless similarity scores.

Stage 2 — Similarity search: The query vector is compared against all stored document embeddings using the `<=>` cosine distance operator in pgvector:

sql

```
SELECT title, content, 1 - (embedding <=> %s::vector) AS similarity
FROM policy_documents
WHERE 1 - (embedding <=> %s::vector) > 0.5
ORDER BY embedding <=> %s::vector
LIMIT 3;
```

The HNSW index (`vector_cosine_ops`) makes this search approximate but fast, avoiding a full table scan on every query.

Stage 3 — Retrieved documents injected into prompt: The top-K most similar policy documents are passed to the LLM as context in the system prompt. The agent in `skeleton/agent.py` constructs the prompt with the retrieved policy content as ground truth.

Stage 4 — LLM generates the answer: The LLM reads the retrieved policy text alongside the user's question and generates a grounded answer based on actual policy content rather than relying on its training data. This prevents hallucination on domain-specific rules (e.g., the exact refund percentages in RF001–RF005).

4.3 Embedding Dimension and Provider Switching

Our implementation uses **768 dimensions** (Ollama `nomic-embed-text`). This is declared in `schema.sql`:

sql

```
embedding vector(768)
```

If the provider is switched to Gemini after seeding, Gemini's embedding model (`gemini-embedding-001`) produces **3072-dimensional** vectors. Attempting to query a 768-dimensional index with a 3072-dimensional query vector will cause a **dimension mismatch error** — pgvector will refuse the query entirely, making the vector index unusable until the database is rebuilt.

The stored embeddings and query embeddings **must always use the same model**. To switch providers safely: change `LLM_PROVIDER` in `.env`, update `schema.sql` to `vector(3072)`, run `docker compose down -v && docker compose up -d` to rebuild the database, and re-run `seed_vectors.py` to regenerate all embeddings with the new model.

Section 5 — AI Tool Usage Evidence

Example 1 — Schema Design for Junction Table

Context: Designing the stop sequence table for metro schedules. The initial instinct was to use a JSONB array column for the ordered list of stops.

Prompt:

> "I need to store the ordered list of stops for a metro schedule in PostgreSQL. The JSON data has a field called stops_in_order which is an array like ['MS01', 'MS02', 'MS09']. Should I store this as a JSONB array or create a separate table? The grading rubric says stops must be in a separate junction table with a stop_order column."

Outcome: Claude confirmed that a junction table is required for 3NF compliance and explained the functional dependency: `(schedule_id, station_id) → stop_order`. The suggested schema matched the rubric requirement exactly. We used this output directly and added the `arrival_time_offset_min` column ourselves after studying the JSON data.

Example 2 — Fixing Circular Foreign Key

Context: `metro_stations` and `national_rail_stations` need to reference each other (interchange stations), but PostgreSQL cannot create both tables simultaneously with mutual FK references.

Prompt:

> "I have two tables that reference each other: metro_stations has a FK to national_rail_stations, and national_rail_stations has a FK to metro_stations. PostgreSQL gives an error because neither table exists when the other is being created. How do I solve this?"

Outcome: Claude suggested creating both tables first without the mutual FKs, then adding them with `ALTER TABLE ADD FOREIGN KEY` afterward. This worked correctly. However, Claude initially suggested using `ADD CONSTRAINT IF NOT EXISTS` which PostgreSQL 16 does not support — we discovered this error during testing and had to remove the `IF NOT EXISTS` clause from the `ADD CONSTRAINT` statement.

Example 3 — AI Output That Was Wrong and Needed Correction

Context: Implementing `query_national_rail_availability`. We asked Claude to write the function.

Prompt:

> "Write a Python function query_national_rail_availability(origin_id, destination_id, travel_date) that returns national rail schedules between two stations. Use psycopg2 with RealDictCursor. The schema has national_rail_schedules and national_rail_schedule_stops tables."

Outcome: Claude generated a function that used a JSONB containment operator (`@>`) assuming stops were stored as a JSONB array — but our schema uses a normalised junction table instead. The generated SQL was:

```
WHERE stops_in_order @> %s::jsonb  -- WRONG: no such column
```

We identified the error by running the function and getting a `column does not exist` error. We corrected the SQL to use a JOIN on `national_rail_schedule_stops` with a `stop_order` comparison instead.

Example 4 — bcrypt Password Hashing Implementation

Context: The grading rubric states that plain-text passwords score 0. We needed to implement bcrypt correctly in both `register_user()` and `seed_postgres.py`.

Prompt:

> "Show me how to hash a password with bcrypt in Python during user registration, and how to verify it during login using psycopg2. The password column in PostgreSQL is VARCHAR(128)."

Outcome: Claude provided correct implementations for both `bcrypt.hashpw()` with `bcrypt.gensalt()` for registration and `bcrypt.checkpw()` for login verification. The output was used directly. We confirmed it worked by running `login_user('alice.tan@email.com', 'alice1990')` and verifying it returned the user dict.

Example 5 — Debugging Seed Script Encoding Error

Context: The seed script had Chinese comments that caused syntax errors when Python tried to parse the file.

Prompt:

> "My queries.py file has a SyntaxError: unterminated triple-quoted string literal. Running python -c 'import ast; ast.parse(open(file).read())' shows the error is at line 857 but that line looks fine. How do I find the actual cause?"

Outcome: Claude suggested counting the total number of `"""` occurrences to check if they are balanced (should be even). The count returned 79 (odd), confirming a missing closing `"""`. Claude then suggested using PowerShell to find all lines containing non-ASCII characters, which revealed that Chinese comment lines had been corrupted into invalid byte sequences that broke the triple-quoted string parsing. The fix was to replace the entire file with a clean English-only version.

Example 6 — Expanding Policy Knowledge Base

Context: The AI assistant required additional policy coverage for refund requests, ticket eligibility, and travel regulations.

Prompt:

> "Create structured JSON policy entries for typhoon refunds, student tickets, senior tickets, and bicycle transport rules that can be retrieved through the RAG system."

Outcome: The generated policy structures were reviewed and adapted to match the project's JSON schema. These additions expanded the knowledge base available to the AI assistant and improved policy-related responses.

Section 6 — Reflection & Trade-offs

6.1 Design Decisions

Decision 1: Natural keys (VARCHAR) over surrogate keys (SERIAL/UUID)

We chose natural keys such as `MS01`, `NR_SCH01`, and `RU01` as primary keys because the JSON source data already uses these stable, human-readable identifiers. Using natural keys eliminates the need for an ID mapping layer and makes queries easier to read and debug (e.g., `WHERE station_id = 'NR01'` is self-explanatory). The trade-off is that if the naming convention ever changes, renaming primary keys is expensive. For a single-region educational system with stable identifiers, natural keys are the correct choice.

Decision 2: Polymorphic association via two nullable FKs with a CHECK constraint

The `payments` table must associate with either a `national_rail_bookings` row or a `metro_trips` row, but not both. We chose to use two nullable FK columns (`booking_id` and `trip_id`) with a `CHECK` constraint enforcing that exactly one is non-null. This preserves full referential integrity at the database level. The alternative — using a single

`transaction_ref` string with a prefix (`BK-` or `MT-`) — is simpler but sacrifices FK enforcement, allowing orphaned payment records if a booking is deleted.

6.2 What Would Be Different in Production

Schema migrations instead of full resets: In development, we reset the database with `docker compose down -v` whenever the schema changes. In production, this would destroy all user data. A production system would use a migration tool such as Alembic or Flyway, where each schema change is a versioned script applied incrementally without data loss.

Connection pooling: Our implementation opens a new `psycopg2` connection for every query function call. Under concurrent load (multiple users simultaneously), this would exhaust PostgreSQL's connection limit (default 100). A production system would use `psycopg2.pool.ThreadedConnectionPool` or an external pooler like PgBouncer to reuse connections across requests.
